

Day 48: Transfer Learning -- Freezing, Fine-Tuning, and Why the Head Matters

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 7, Day 7

Day 47 recreated a real historical architecture, LeNet-5, and honestly found it wasn't automatically the right tool for this series' own small dataset. Today moves from designing your own architecture into reusing someone else's already-trained one: a network is pretrained on the usual junction dataset, then a real visually-shifted target domain is built with only 20 labeled images -- a realistic low-data scenario. Five honestly-run strategies are compared on those same 20 images, and the real result is more interesting than a simple 'freeze helps' story: training from scratch on 20 images is WORSE than doing nothing at all, and the two strategies that actually win both share one thing -- they keep the source network's head instead of discarding it.

0. Where Today Fits

Start here (no prior knowledge assumed): Every network trained in this series so far -- Days 43 through 47 -- has been trained from scratch, every single time, on its own target task. Today asks a different question: if you already have a network trained on a RELATED task, how should you reuse it on a new task where you only have a handful of labeled examples? This is transfer learning: reusing what a network already learned instead of relearning it from zero.

1. The Setup: A Source Network and a Data-Starved Target

Start here (no prior knowledge assumed): A 'source' network is trained the normal way -- this series' own TwoBlockConvNetRGB, on the usual junction dataset (arm=8, 200 images) -- exactly like every prior day. Then a 'target' domain is built with a REAL visual shift: longer arms (11 vs. 8), doubled color noise (0.30 vs. 0.15), and more position jitter (3 vs. 2). Only 20 labeled target images (5 per class) are made available for training -- a realistic low-data scenario -- scored on a separate, never-trained-on 80-image target test set.

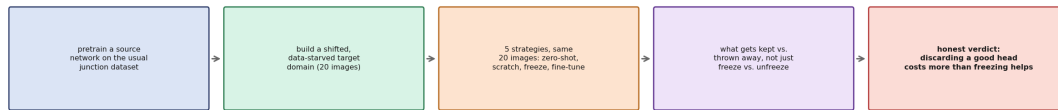
Topic specification: Source: TwoBlockConvNetRGB (2,024 params), arm=8, noise=0.4, channel_noise=0.15, center_jitter=2, n_per_class=50 (200 images), seed=42, 400 epochs, lr=0.08 -- identical to this series' baseline recipe since Day 43. Target: arm=11, noise=0.4, channel_noise=0.3, center_jitter=3; train n_per_class=5 (20 images, seed=123); test n_per_class=20 (80 images, seed=777).

Explanation: The target domain keeps the exact same 4-class task (how many arms radiate from a center point) but looks visibly different -- bigger, noisier, more jittered. This mirrors a realistic transfer scenario: same underlying problem, different visual conditions, and nowhere near enough labeled data in the new domain to train a network from scratch and expect it to generalize.

Real-world scenario: A quality-control camera trained to spot defects under one factory's lighting rig gets moved to a second factory with different lighting, a different camera angle, and only 20 labeled example photos so far from the new site. Retraining from scratch on 20 images would almost certainly fail to generalize -- but the first factory's model already learned useful, transferable visual features.

How it works, visually:

Day 48 pipeline: pretrain, shift the domain, starve the data, compare 5 honest strategies



Today's full pipeline: pretrain, shift the domain, starve the data, compare 5 honest strategies.

Watch: [What is Pretrained Models | Transfer learning in deep learning | DataMites](#) — a general introduction to pretrained models and why transfer learning saves data and training time.

2. Freezing -- New From-Scratch Code

Start here (no prior knowledge assumed): Freezing a layer means: keep running it forward every step (its output is still needed), but never update its weights. A frozen layer's forward pass still runs on every training step -- it's not skipped -- but its own weights never move away from wherever they started.

Topic specification: `backward_transfer(net, y_onehot, freeze_conv)`: always computes and applies the head's gradients (`dW1`, `db1`, `dW2`, `db2`). Only when `freeze_conv=False` does it ALSO compute and apply the conv layers' gradients via `maxpool_backward` + `conv2d_backward_mc`.

Explanation: Because `TwoBlockConvNetRGB`'s conv layers are the very FIRST layers in the network -- there is nothing earlier that also needs a gradient -- freezing them means `backward_transfer` can skip computing `conv2d_backward_mc` and `maxpool_backward` for them ENTIRELY when `freeze_conv=True`, not just skip their final weight update. This is specific to freezing input-adjacent layers: freezing a layer in the MIDDLE of a deeper network would still require propagating gradient through it to reach the earlier layers, even though that middle layer's own weights wouldn't change.

Mathematics:

MATH

freeze_conv=True: update only

$$W1 \leftarrow lr \cdot dW1, \quad b1 \leftarrow lr \cdot db1, \quad W2 \leftarrow lr \cdot dW2, \quad b2 \leftarrow lr \cdot db2$$

freeze_conv=False: ALSO update

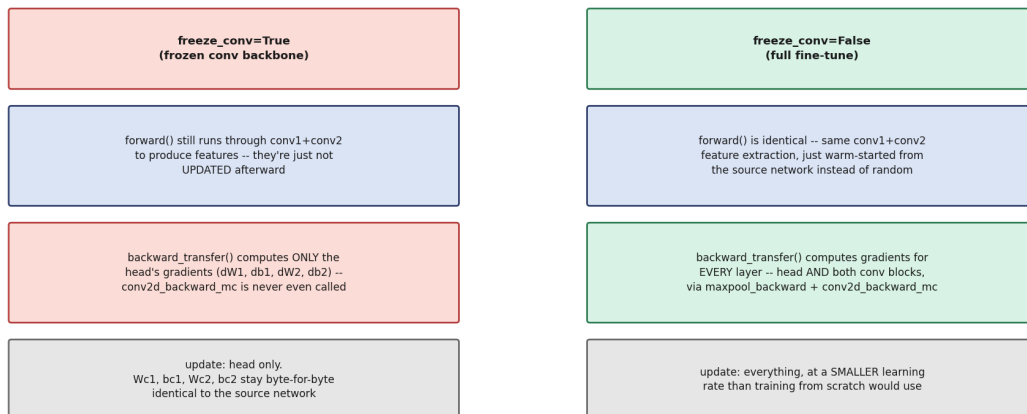
$$Wc1 \leftarrow lr \cdot dWc1, \quad bc1 \leftarrow lr \cdot dbc1, \quad Wc2 \leftarrow lr \cdot dWc2, \quad bc2 \leftarrow lr \cdot dbc2$$

In both cases, $dW1$ and $dW2$ are computed identically -- freezing only changes what happens AFTER the head's own gradients are found.

Implementation:

```
def backward_transfer(net, y_onehot, freeze_conv):
    m = y_onehot.shape[0]
    dZ2 = net.probs - y_onehot
    dW2 = net.A1.T @ dZ2 / m
    db2 = dZ2.sum(axis=0) / m
    dA1 = dZ2 @ net.W2.T
    dZ1 = dA1 * dtanh(net.Z1)
    dW1 = net.flat.T @ dZ1 / m
    db1 = dZ1.sum(axis=0) / m
    updates = [(net.W2, dW2), (net.b2, db2),
               (net.W1, dW1), (net.b1, db1)]
    if not freeze_conv:
        dflat = dZ1 @ net.W1.T
        dpooled2 = dflat.reshape(net.pooled2.shape)
        dAc2 = maxpool_backward(dpooled2, net.Ac2, net.masks2, 2, 2)
        dZc2 = dAc2 * drelu(net.Zc2)
        dWc2, dbc2, dpooled1 = conv2d_backward_mc(
            dZc2, net.pooled1, net.Wc2)
        dAc1 = maxpool_backward(dpooled1, net.Ac1, net.masks1, 2, 2)
        dZc1 = dAc1 * drelu(net.Zc1)
        dWc1, dbc1, _ = conv2d_backward_mc(dZc1, net.X, net.Wc1)
        updates += [(net.Wc1, dWc1), (net.bc1, dbc1),
                   (net.Wc2, dWc2), (net.bc2, dbc2)]
    for p, g in updates:
        p -= net.lr * g
```

How it works, visually:



The same backward_transfer() call, freeze_conv=True vs. False.

Watch: Freezing vs. Unfreezing Layers: The Ultimate Guide to Transfer Learning — covers exactly today's core mechanic -- which layers to freeze, which to unfreeze, and why.

Why choose this? Freezing early layers saves real compute (their backward pass is skipped entirely, not just their update), protects already-good features from being damaged by too few new examples, and is simple to reason about -- the conv weights are byte-for-byte guaranteed unchanged.

Why NOT this (tradeoffs)? Freezing is only as good as what it protects. Today's own results show that freezing the conv layers while discarding a well-trained head can perform WORSE than doing no transfer at all -- freezing protects the features, but says nothing about whether the rest of the network is being set up to succeed.

3. Gradient-Checking the New Component

Start here (no prior knowledge assumed): Day 44 found a numerical gradient check can pass on an isolated function tested with its own ad hoc loss, yet still hide a real bug. The fix, followed again today: check backward_transfer through the exact forward()->loss->backward_transfer() pipeline training actually uses, in BOTH freeze_conv modes, on a small input.

Explanation: freeze_conv=False is checked against all four weight tensors (W1, W2, Wc1, Wc2) -- it should reduce to the same math as this series' own TwoBlockConvNetRGB.backward(), verified independently rather than assumed. freeze_conv=True is checked against only the two head tensors (W1, W2), since the conv gradients are never computed at all in that mode. A separate, direct check then confirms freeze_conv=True doesn't just skip computing conv gradients in theory -- the conv weight ARRAYS themselves are byte-for-byte unchanged after 20 real training epochs.

Actual output:

```
=====
1. GRADIENT CHECK -- backward_transfer, both freeze_conv modes
=====
freeze_conv=False  checked ['W1', 'W2', 'Wc1', 'Wc2']  max_rel_err=2.38e-08  (PASS)
freeze_conv=True   checked ['W1', 'W2']               max_rel_err=5.39e-09  (PASS)

Direct check that freeze_conv=True literally freezes the conv weights
(not just skips their gradient in principle):
Wc1 unchanged after 20 epochs: True
Wc2 unchanged after 20 epochs: True
```

Actual output from this lesson's run.

Term refresher — relative error: $|numeric - analytic| / (|numeric| + |analytic|)$ -- used instead of plain absolute difference because it stays meaningful whether the gradient itself is large or tiny. Both freeze_conv modes pass far below the roughly $1e-4$ threshold that would flag a real problem.

4. Pretraining the Source Network

Start here (no prior knowledge assumed): Before any transfer strategy can be tried, there has to be something to transfer FROM. The source network is trained with this series' exact standard recipe -- nothing new here, just a normal Day-43-style training run that produces the pretrained weights every strategy below will reuse.

Actual output:

network	params	train_acc	test_acc
source (TwoBlockConvNetRGB)	2,024	1.0000	0.8500

Actual output from this lesson's run (source domain: arm=8, its own held-out test set).

Explanation: This is the exact same train_acc/test_acc this series' baseline has reached since Day 43 -- expected, since it's the identical recipe. What's new today is what happens next: this trained

network becomes the STARTING POINT for five different strategies on a domain it was never trained on.

5. Five Honest Strategies, Same 20 Target Images

Start here (no prior knowledge assumed): Every strategy below sees the IDENTICAL 20 target-domain training images and is scored on the IDENTICAL 80-image target test set. The only thing that differs between them is what gets copied from the source network, what gets frozen, and what gets trained.

Topic specification: zero-shot: source network, no target training at all. from-scratch: random init, ignores the source entirely, trains all layers. frozen+fresh head: conv frozen from source, head reinitialized randomly, only head trains. frozen+warm head: conv frozen from source, head ALSO kept from source, only head continues training. fine-tune: every layer warm-started from source, every layer keeps training, at a REDUCED learning rate.

How it works, visually:

Same 20 target images, 5 strategies: what's kept vs. thrown away vs. trained			
strategy	conv layers	head	target test_acc
1. zero-shot	kept from source (no training at all)	kept from source (no training at all)	0.6375
2. from-scratch	random init, TRAINED	random init, TRAINED	0.3625 (worst)
3. frozen + fresh head	frozen from source	random init, TRAINED	0.4500
4. frozen + warm head	frozen from source	kept from source, TRAINED	0.6625 (best, tied)
5. fine-tune (lr=0.01)	kept from source, TRAINED	kept from source, TRAINED	0.6625 (best, tied)

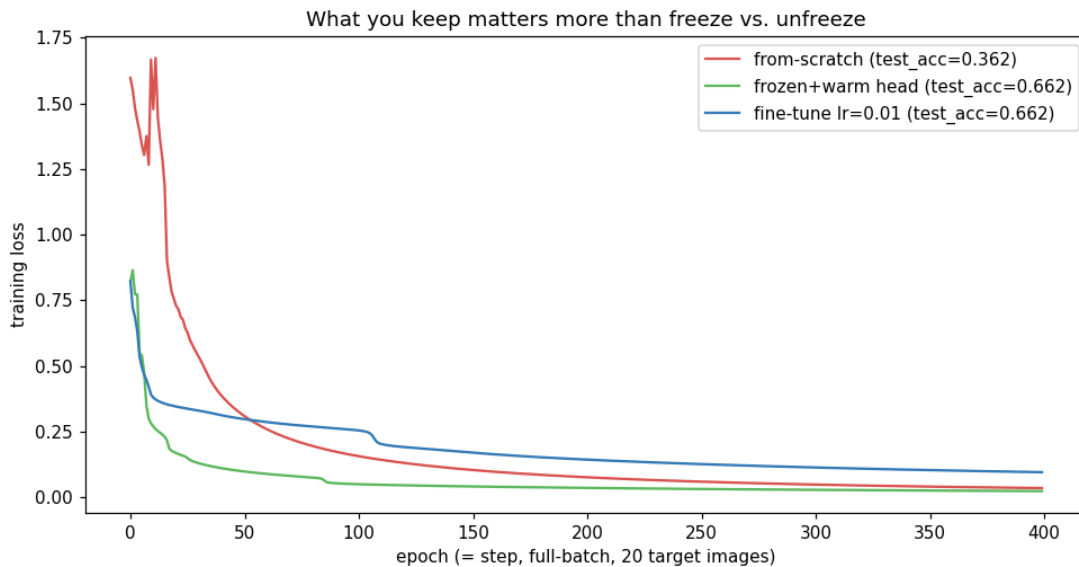
What each of the 5 strategies keeps, discards, and trains -- and the real test_acc each one reached.

Actual output:

strategy	train_acc	test_acc
zero-shot	0.8000	0.6375
from-scratch	1.0000	0.3625
frozen + fresh head	0.8500	0.4500
frozen + warm head	1.0000	0.6625
fine-tune (lr=0.01)	1.0000	0.6625

Actual output from this lesson's run.

How it works, visually:



Training loss on the same 20 target images -- from-scratch, frozen+warm head, and fine-tune (lr=0.01).

Explanation: Training loss alone doesn't tell the full story here: frozen+warm head reaches the LOWEST final training loss of the three plotted curves, and fine-tune's loss settles noticeably higher -- yet both reach the exact same test_acc=0.6625. Training loss measures fit to the 20 training images; test_acc measures what actually generalizes. On a dataset this small, they can and do disagree.

Why choose this? When you have a related pretrained network and very little target data, reusing what already works (a good feature extractor, a well-calibrated head, or both) consistently beats training from scratch -- today's from-scratch result is the worst of all five strategies by a wide margin.

Why NOT this (tradeoffs)? 'Freeze the early layers' is not, by itself, a guarantee of success. Frozen+fresh head -- often the textbook-default transfer recipe -- actually UNDERPERFORMED doing nothing at all (zero-shot) here, because discarding a well-calibrated head and re-learning one from 20 examples lost more than freezing the features gained.

6. Why the Head Matters More Than Freeze vs. Unfreeze

Start here (no prior knowledge assumed): Look across all five strategies: the two that beat zero-shot (frozen + warm head, and fine-tune) both KEEP the source network's head. The two that lose to zero-shot (from-scratch, and frozen + fresh head) both THROW AWAY the source head and relearn one from only 20 examples. Whether the conv layers were frozen or not doesn't sort the results nearly as cleanly as this does.

Actual output:

fine-tune lr sweep	train_acc	test_acc
0.08	0.9500	0.5250
0.03	1.0000	0.6375
0.01	1.0000	0.6625
0.005	0.9500	0.6250

Actual output from this lesson's run.

Explanation: The learning-rate sweep shows a second real finding: fine-tuning at the SAME learning rate used to train from scratch (0.08) actually underperforms zero-shot (test_acc=0.5250 vs. 0.6375) -- large, confident gradient steps computed from only 20 noisy examples partially overwrite the source network's already-good weights before those 20 examples can teach it anything reliably better. Lowering the learning rate to 0.01 lets fine-tuning nudge the pretrained weights gently enough to actually help, reaching the best result tied with frozen+warm head. A learning rate that works well for training from scratch is not automatically a safe choice for fine-tuning a pretrained network.

Term refresher — catastrophic forgetting: When continued training on new data overwrites useful patterns a network already learned, instead of building on them. Today's lr=0.08 fine-tune result is a mild, real instance of this: enough of the source network's useful weights got disturbed by large updates from just 20 examples that the result ended up worse than not training at all.

7. What This Maps To in PyTorch

Start here (no prior knowledge assumed): Every piece of today's freeze/fine-tune machinery has a direct PyTorch equivalent -- the `requires_grad` flag is the framework's version of today's `freeze_conv` branch.

Implementation:

```
model = MyConvNet()
model.load_state_dict(torch.load('source_weights.pt'))

# freeze the conv backbone -- matches freeze_conv=True above
for p in model.conv1.parameters(): p.requires_grad = False
for p in model.conv2.parameters(): p.requires_grad = False

# option A: fresh head (matches 'frozen+fresh head')
model.fc2 = nn.Linear(model.fc2.in_features, n_classes)

# option B: keep the pretrained head (matches 'frozen+warm head')
# -- just don't touch model.fc2 at all

optimizer = torch.optim.SGD(
    filter(lambda p: p.requires_grad, model.parameters()), lr=0.08)

# full fine-tune needs every param trainable AND a smaller
# learning rate than training from scratch would use:
for p in model.parameters(): p.requires_grad = True
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
```

Explanation: `filter(lambda p: p.requires_grad, ...)` is PyTorch's equivalent of today's `updates` list inside `backward_transfer` -- the optimizer only ever steps parameters still marked trainable, exactly like today's code only appends the conv layers' gradients to `updates` when `freeze_conv` is False.

Interview Preparation — Technical Questions

Q: Why can `backward_transfer` skip computing conv gradients entirely when `freeze_conv=True`, rather than just skipping their final update?

A: Because the conv layers are the very FIRST layers in this network -- there is nothing earlier that also needs a gradient computed through them. If a frozen layer were in the middle of a deeper network, gradient would still need to flow through it (via the chain rule) to reach earlier trainable layers, even though the frozen layer's own weights wouldn't change.

Q: Frozen+fresh head reached `test_acc=0.4500`, worse than zero-shot's 0.6375. Why would ADDING training make things worse?

A: The source network's original head had already learned a well-calibrated decision boundary. Reinitializing it randomly and retraining it on only 20 examples can't relearn as good a boundary as the one that was thrown away -- the frozen conv features are reused, but the thing making the final decision is worse than before.

Q: Why does fine-tuning at `lr=0.08` (`test_acc=0.5250`) underperform fine-tuning at `lr=0.01` (`test_acc=0.6625`), when 0.08 is the same learning rate used to train the source and from-scratch networks successfully?

A: 0.08 was tuned for training a randomly-initialized network from scratch on 200 images, where large, aggressive steps are reasonable early on. Applied to a WARM-STARTED network on only 20 examples, that same step size partially overwrites already-good pretrained weights before the tiny dataset can reliably teach it anything better -- a mild case of catastrophic forgetting.

Q: What does 'warm-started' mean, and which of today's five strategies use it?

A: Warm-started means a network's weights are initialized by copying them from an already-trained network, instead of the usual random initialization. Frozen+fresh head warm-starts only the conv layers; frozen+warm head and fine-tune warm-start every layer, conv and head alike; from-scratch warm-starts nothing.

Q: Both frozen+warm head and fine-tune (`lr=0.01`) reach the identical `test_acc=0.6625`. Are they doing the same thing?

A: No -- frozen+warm head only ever updates the head (4 tensors), while fine-tune updates all 8 tensors including both conv layers. They tie on final accuracy here, but frozen+warm head does it with less compute (no conv backward pass at all) and less risk of disturbing the pretrained features, since the conv weights literally cannot change.

Interview Preparation — Theoretical Questions

Q: Why might training from scratch on very little data (20 images) end up WORSE than not training at all (zero-shot)?

A: With only 20 examples, a randomly-initialized network has nowhere near enough signal to learn features that generalize -- it can easily reach perfect training accuracy by memorizing those 20 images specifically, without learning anything transferable about the underlying class structure. A network that was never trained on the target domain at all, but WAS trained on a related domain with 200 images, can end up with more useful, generalizable features purely because it saw more real signal, even if that signal came from a slightly different-looking dataset.

Q: Is freezing early layers always safe, and unfreezing them always risky, regardless of how much target data you have?

A: No. Freezing protects already-good features from being damaged, which matters most when target data is scarce (as it is today). With enough target data, fine-tuning every layer at an appropriately reduced learning rate can outperform freezing, because the network gets to adapt its features to the new domain rather than staying locked to the source domain's exact visual statistics.

Q: What's the general principle behind choosing a smaller learning rate for fine-tuning than for training from scratch?

A: A freshly initialized network's weights carry no useful information yet, so large, confident gradient steps are appropriate early in training. A warm-started network's weights already encode real, useful structure -- large steps risk overwriting that structure faster than a small amount of new data can teach the network something reliably better to replace it with.

Key Takeaway and What's Next

Key takeaway: Freezing conv layers when they're the network's first layers lets backward_transfer skip their gradient computation entirely, not just their update (gradient-checked to 5.39e-09 relative error, with a direct confirmation that frozen weights are byte-for-byte unchanged after training). On a real, honestly-run transfer experiment -- a source network pretrained normally, then tested against a visually shifted target domain with only 20 labeled images -- training from scratch was the WORST strategy (test_acc=0.3625), even worse than using the source network completely unmodified (zero-shot, test_acc=0.6375). Freezing the conv layers while discarding the head also underperformed zero-shot (test_acc=0.4500). The two strategies that actually won -- frozen+warm head and a properly-tuned full fine-tune at lr=0.01, both reaching test_acc=0.6625 -- share one property: neither discarded a part of the source network that already worked. The lesson isn't simply 'freeze early layers, it helps' -- it's that with very little target data, throwing away learned structure (whether features OR a decision boundary) costs more than freezing protects, and if you do keep training every layer, the learning rate that worked for training from scratch is not automatically safe for fine-tuning.

Suggested watch order, if starting from zero

- 1. "What is Pretrained Models | Transfer learning in deep learning | DataMites" -- the general concept this entire lesson builds on.
- 2. "Freezing vs. Unfreezing Layers: The Ultimate Guide to Transfer Learning" -- today's core mechanic, freeze vs. fine-tune, in more depth.

Day 49 preview

- AlexNet-style deeper CNN -- a deeper network with ReLU, dropout, and larger images, trained with mini-batches for the first time in this series (every network so far has trained on its full dataset in one batch per epoch).